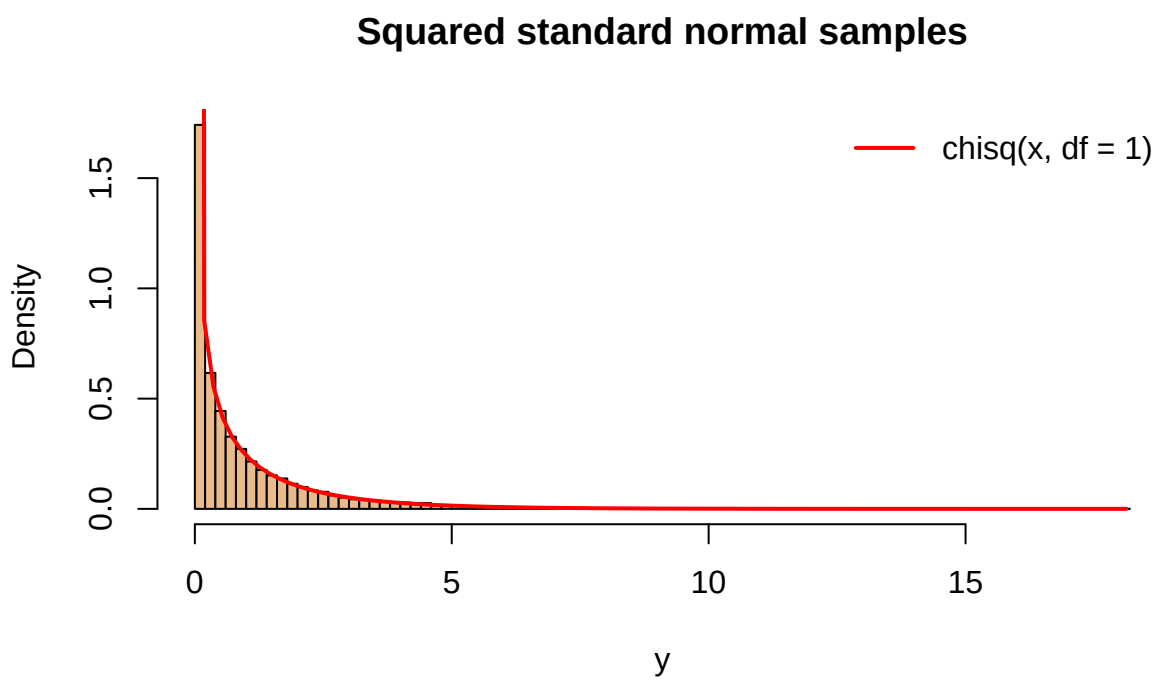
STAT6011 Assignment 1

Yu Boyang / ID: 3030144197

Question 1

(a)

```
set.seed(6001)
x <- rnorm(10000)
y <- x^2
hist(y, breaks = 100, probability = TRUE, col = "#e9bb8a",
     main = "Squared standard normal samples", xlab = "y")
curve(dchisq(x, df = 1), from = 0.0001, to = max(y),
      add = TRUE, col = "red", lwd = 2)
legend("topright", "chisq(x, df = 1)", col = "red", lwd = 2, bty = "n")
```



The histogram is very similar and very close to the chi-squared(df=1) density.

(b)

Use a one-sample Kolmogorov–Smirnov test with $H_0 : Y \sim \chi^2_1$.

```
ks <- ks.test(y, "pchisq", df = 1)
ks
```

```
##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  y
## D = 0.00647, p-value = 0.8
## alternative hypothesis: two-sided
```

At the 5% level, do not reject H_0 (p-value = 0.7972).

Question 2

(a)

```
h <- function(x) (cos(50*x) + sin(20*x))^2
set.seed(6001)
n <- 10000
u <- runif(n)
mc <- mean(h(u))
mc.se <- sd(h(u))/sqrt(n)
c(estimate = mc, MCSE = mc.se)
```

```
## estimate     MCSE
## 0.958516 0.010379
```

(b)

For $U_i \sim \text{Beta}(\alpha, \beta)$,

$$\widehat{I} = \frac{1}{n}\sum_i \frac{h(U_i)}{f(U_i)}, \qquad \text{Var}(\widehat{I}) = \frac{1}{n}\left[\int_0^1 \frac{h(x)^2}{f(x)}dx - I^2\right].$$

Minimize $\int_0^1 h(x)^2/f(x)dx$ over $0 < \alpha, \beta < 2$.

```
second.moment <- function(ab) {
  integrate(function(x) h(x)^2/dbeta(x, ab[1], ab[2]),
            0, 1, subdivisions = 1000)$value
}
opt <- optim(c(1, 1), second.moment, method = "L-BFGS-B",
```

```
            lower = c(.01, .01), upper = c(1.99, 1.99))
ab <- opt$par
names(ab) <- c("alpha", "beta")
ab
```

```
##  alpha   beta
## 1.0780 1.0145
```

```
set.seed(6001)
u.beta <- rbeta(n, ab[1], ab[2])
z <- h(u.beta)/dbeta(u.beta, ab[1], ab[2])
is <- mean(z)
is.se <- sd(z)/sqrt(n)
c(estimate = is, MCSE = is.se)
```

```
## estimate     MCSE
## 0.964754 0.010377
```

The optimal values are $\alpha \approx 1.078$ and $\beta \approx 1.0145$.

(c)

Use Gauss–Legendre quadrature with $x = (t+1)/2$:

$$I \approx \frac{1}{2} \sum_k w_k h((t_k+1)/2).$$

```
gauss.int <- function(m) {
  g <- statmod::gauss.quad(m, kind = "legendre")
  sum(g$weights*h((g$nodes + 1)/2))/2
}
benchmark <- gauss.int(128)
benchmark
```

```
## [1] 0.9652
```

```
comparison <- data.frame(
  method = c("Uniform MC", "Beta importance sampling", "Gauss-Legendre"),
  estimate = c(mc, is, benchmark),
  MCSE = c(mc.se, is.se, NA))
comparison$absolute.error <- abs(comparison$estimate - benchmark)
knitr::kable(longtable = FALSE, comparison, digits = 7)
```

method	estimate	MCSE	absolute.error
Uniform MC	0.95852	0.010379	0.0066852
Beta importance sampling	0.96475	0.010377	0.0004466
Gauss-Legendre	0.96520	NA	0.0000000

```r
var.uniform <- second.moment(c(1, 1)) - benchmark^2
var.beta <- second.moment(ab) - benchmark^2
c(variance.ratio = var.beta/var.uniform,
  variance.reduction = 1 - var.beta/var.uniform)
```

```
##     variance.ratio variance.reduction
##          0.9930797          0.0069203
```

The integral is about 0.9652. The Beta proposal reduces variance by about 0.69%.

Question 3

(a)

```r
d <- read.csv("Q3.csv", row.names = 1)
fit <- lm(y ~ ., data = d)
X <- model.matrix(fit)
y3 <- d$y
n3 <- nrow(X)
p <- ncol(X)
bhat <- coef(fit)
S <- sum(resid(fit)^2)
vhat <- S/n3
log.Lmax <- -n3/2*(log(2*pi*vhat) + 1)
c(sigma2.MLE = vhat, log.Lmax = log.Lmax)
```

```
## sigma2.MLE   log.Lmax
##     0.8004  -130.7617
```

Use $\widehat{\beta} = (X^T X)^{-1} X^T y$ and $\widehat{\sigma}^2 = S/n$. Accept a prior sample with probability $L(\beta, \sigma^2)/L(\widehat{\beta}, \widehat{\sigma}^2)$.

```r
prior.reject <- function(prior) {
  accepted <- 0
  for (i in 1:100) {
    if (prior == "normal") {
      b <- matrix(rnorm(10000*p, sd = 5), nrow = 10000)
    } else {
      b <- matrix(rexp(10000*p)*sample(c(-1, 1), 10000*p, TRUE),
                  nrow = 10000)
    }
    v <- 1/rgamma(10000, .1, rate = .1)
    rss <- colSums((y3 - X %*% t(b))^2)
    log.a <- n3/2*(log(vhat/v) + 1) - rss/(2*v)
    accepted <- accepted + sum(log(runif(10000)) <= log.a)
  }
  accepted
}
```

```r
set.seed(6001)
prior.reject("normal")
```

```
## [1] 0
```

So tried 1,000,000 proposals but got no accepted samples, just like the student on moodle stated.

(b)

```r
prior.reject("laplace")
```

```
## [1] 0
```

The Laplace prior also gave no accepted samples after 1,000,000 proposals, so could not compare their efficiency from these runs.

(c)

Fit ridge and Lasso using 10-fold cross-validation.

```r
set.seed(6001)
fold <- sample(rep(1:10, length.out = n3))
x3 <- as.matrix(d[, paste0("x", 1:10)])
cv.ridge <- glmnet::cv.glmnet(x3, y3, alpha = 0, foldid = fold)
cv.lasso <- glmnet::cv.glmnet(x3, y3, alpha = 1, foldid = fold)
knitr::kable(longtable = FALSE, data.frame(parameter = c("beta0", paste0("beta", 1:10)),
  MLE = bhat,
  Ridge.CV = as.numeric(coef(cv.ridge, s = "lambda.min")),
  Lasso.CV = as.numeric(coef(cv.lasso, s = "lambda.min"))),
  digits = 4, row.names = FALSE)
```

parameter	MLE	Ridge.CV	Lasso.CV
beta0	1.1671	1.1774	0.9455
beta1	0.9791	0.9132	0.9321
beta2	0.6033	0.5606	0.5686
beta3	-0.0505	-0.0713	0.0000
beta4	0.0517	0.0301	0.0000
beta5	0.3775	0.3877	0.2776
beta6	0.1162	0.0967	0.0576
beta7	0.0336	0.0451	0.0000
beta8	-0.4191	-0.4507	0.0000
beta9	-0.4019	-0.4120	-0.3381
beta10	0.2154	0.1932	0.1274

```r
c(ridge.lambda = cv.ridge$lambda.min, lasso.lambda = cv.lasso$lambda.min)
```

```
## ridge.lambda lasso.lambda
##     0.104269     0.053116
```

Ridge shrinks the coefficients, while Lasso sets some coefficients to zero.

Question 4

(a)

For $\mu \sim N(m, 4)$,

$$p(\mu \mid y) \propto \exp\left[-\frac{1}{2}\left\{\sum_i (y_i - \mu)^2 + \frac{(\mu - m)^2}{4}\right\}\right].$$

Completing the square,

$$\mu \mid y \sim N\left(\frac{n\bar{y} + m/4}{n + 1/4}, \frac{1}{n + 1/4}\right).$$

Set $m = 0$ for the given prior.

```r
y4 <- read.csv("Q4.csv", row.names = 1)$y
n4 <- length(y4)
ybar <- mean(y4)
post.var <- 1/(n4 + 1/4)
c(MLE = ybar, posterior.mean = n4*ybar*post.var,
  posterior.variance = post.var)
```

```
##                MLE     posterior.mean posterior.variance
##          0.8420000          0.8399002          0.0099751
```

(b)

The mean update is

$$m_{j+1} = \frac{n\bar{y} + m_j/4}{n + 1/4}, \qquad m_{j+1} - \bar{y} = \frac{m_j - \bar{y}}{4n + 1}.$$

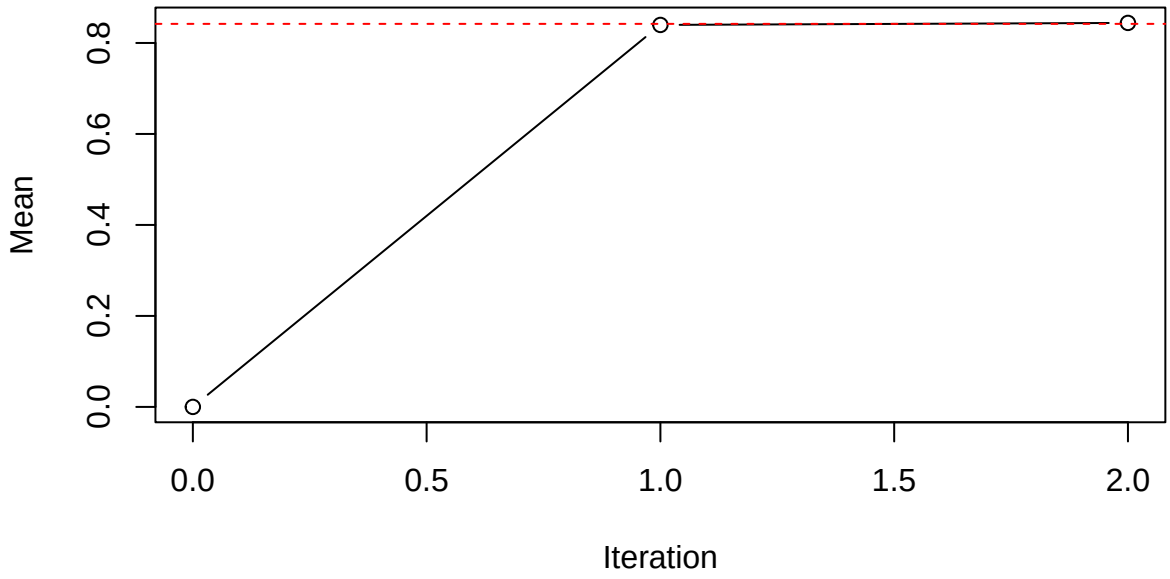
Thus the fixed point is $\bar{y}$. Use 1000 samples per iteration and stop when the mean changes by less than 0.01.

```r
set.seed(6001)
m <- 0
trace <- m
for (j in 1:100) {
  post.mean <- (n4*ybar + m/4)*post.var
  draws <- rnorm(1000, post.mean, sqrt(post.var))
  m.new <- mean(draws)
```

```
  change <- abs(m.new - m)
  m <- m.new
  trace <- c(trace, m)
  if (change < .01) break
}
c(PPMM = m, MLE = ybar)
```

```
##     PPMM      MLE
## 0.84408 0.84200
```

```
plot(0:j, trace, type = "b", xlab = "Iteration", ylab = "Mean")
abline(h = ybar, col = "red", lty = 2)
```



The PPMM estimate is 0.84408, and the MLE is 0.842, saying they are close.